danielflugger.com

Sovereign Publishing Stack · Builder's Addendum

Reference document for personal build, stress-testing, and forward planning

March 2026 · Companion to Platform Plan v6

> *This document is for the builder, not the buyer. It covers how to build and stress-test this stack for yourself first, what to document as you go for patent and authority purposes, how to think about the private offline layer, and how future versions of the product (compliance, DMG, licensing) grow organically from this foundation. Read it alongside v6. Write in it constantly.*

A. Build It For Yourself First — You Are the Super User

The only way to stress-test this system honestly is to run it yourself, under real conditions, for real stakes. Every decision in the product was made because it was the right decision for your own stack first. That is the credential. The documentation that follows from building it for yourself is also the authority signal — the GitHub commits, the honest READMEs, the posts about what broke, the dated record of having figured this out before anyone paid you to show them how.

The super user is not a beta tester. The super user is the person whose name is on the domain, who runs the stack daily, who hits the edge cases first, and who documents them in a private repo before they become public knowledge. That person is you. Everything else — the product, the sessions, the annual health checks, the eventual DMG — is downstream of the depth of your own installation.

Your private repo is the product's R&D department.
Every failure mode you document becomes hardening for the next buyer.

A.1 The Private Build Repository

Create a private GitHub repo: danielflugger/sovereign-stack-build. This is not the product repo. It is your personal engineering log. Every command that doesn't work the first time, every configuration decision and why, every Ghost update that required a workaround, every time Mailgun did something unexpected. Raw, unpolished, chronological. The commit history is the proof of work.

Folder	What it contains
install-logs/	Dated Markdown files: every install command run, output captured, failures noted

failure-modes/	One file per distinct failure: what happened, what caused it, exact fix applied
decisions/	Architecture decision log: why SQLite over Cloud SQL, why Lemon Squeezy over Stripe, why Brevo over Mailgun in a specific scenario. Reasoning dated.
ghost-updates/	Log of every ghost update run: version before, version after, anything that required manual intervention
ollama-experiments/	Model comparisons, query patterns that work, corpus size vs quality observations
louvain-runs/	Weekly graph outputs as JSON, observations on cluster evolution over time
payment-testing/	Both Stripe and Lemon Squeezy tested end to end. Edge cases: failed payments, subscription cancellations, EU VAT edge cases, refunds
email-testing/	Mailgun and Brevo both tested. Deliverability logs, DKIM/SPF verification notes, first send from custom domain

> *This repo never becomes public. It becomes the source material for the product documentation, the failure mode appendix in the buyer guide, and eventually the hardened shell script that becomes the DMG. Its value is in being unfiltered.*

A.2 The Monthly Stress Test Cycle

Run this monthly, every month, without exception. The goal is to be the person who has run ‘ghost update’ forty times before the buyer runs it once.

Week 1	ghost update — capture output, note version change, verify site loads, check email delivery after update
Week 2	Test the provenance layer — publish a draft post, verify GPG-signed commit fires, check timestamp in private repo
Week 3	Ollama query session — run 5–10 queries against private corpus, note which return strong results vs noise, adjust Obsidian tagging if needed
Week 4	Louvain run — review cluster output, note any topic drift, identify the post the graph is asking for
Monthly	Cloud Storage backup verification — confirm last nightly backup exists and is non-zero bytes. Test restore on a throwaway VM once per quarter.

B. Documentation as Authority — The GitHub and README Standard

The honest GitHub README is one of the highest-leverage publishing surfaces available right now. LLMs cite well-structured public repos. Technical buyers discover through them. And unlike a blog post, a README with a commit history has a verifiable timestamp that predates any competitor who publishes the same idea later.

Every public repo in the sovereign stack should follow the same README standard. Not because it looks professional — because it makes the repo citable, discoverable, and attributable.

B.1 The README Standard for Every Public Repo

Section	Content
Problem statement	One paragraph. Exactly what this solves and who has that problem. Written for the person who arrived via an LLM recommendation, not someone browsing GitHub.
What you get	Specific output. Not ‘a better workflow’ — ‘a GPG-signed Ghost publishing stack on GCP with a local Ollama intelligence layer, deployable in a weekend.’
Architecture	One diagram or clear component list. No marketing language.
Why these choices	Brief decision rationale: why Ghost over Substack, why SQLite over Cloud SQL, why Lemon Squeezy over Stripe. Honest, dated reasoning. This is the authority signal.
What breaks	The failure modes section. The three most common things that go wrong and exactly how to fix them. This is what no tutorial provides and what makes the repo genuinely useful.
Outcomes	Your own production metrics where applicable. Honest numbers, not aspirational ones.
Full system	One line linking to danielflugger.com/system. One line noting the advisory slot if relevant.

> *The failure modes section is the most important part of any README in this stack. It is the section that gets cited, shared, and bookmarked. It is also the section that demonstrates you have actually run the system, not just designed it. Write it in real time as you hit the failures.*

B.2 Commit Message Standards for the Provenance Layer

The GPG-signed commit message format for the provenance sidecar. Consistent format matters because these commits are the audit trail — they need to be parseable by a future script, queryable by a future LLM, and legible to a future you trying to reconstruct a timeline.

Field	Format and purpose
Subject line	[PUBLISH] Post title | YYYY-MM-DD HH:MM UTC
Ghost post ID	ghost-id: {uuid} — enables future lookups against Ghost Content API
Canonical URL	url: https://yourdomain.com/slug — the permanent record of where this lives
Content hash	sha256: {hash of post content at time of publish} — tamper detection
GPG signature	Auto-appended by git. The cryptographic timestamp.
Tags	tags: tag1, tag2 — enables Louvain graph to include this node

This commit format means every published post has a machine-readable record: what it was, when it was published, where it lives, what it contained, and a cryptographic proof of authorship. That is the VERA verification thesis applied to a personal publishing workflow. It is also the foundation of any future compliance or licensing argument about your intellectual output.

C. Payment Architecture — Both Stripe and Lemon Squeezy

The product guide recommends Lemon Squeezy as the default for new solo operators because the merchant-of-record model handles tax complexity automatically. But market reality is that Stripe is more recognized, and some buyers will specifically ask for it. The right answer is: build with both, default to Lemon Squeezy, offer Stripe as an option, document the trade-offs honestly.

C.1 Why Both, Not Either

Scenario	Right choice
First income layer, no existing payment setup, international buyers expected	Lemon Squeezy — merchant of record, EU VAT handled automatically, lower compliance overhead
Already have Stripe, US-only buyers expected, comfortable with tax compliance	Stripe — lower take rate, widely recognized, direct relationship

B2B buyer who wants to pay via invoice	Stripe — better B2B invoicing infrastructure
Product buyer who wants to pay via PayPal or Klarna	Lemon Squeezy — broader payment method support
Advisory retainer billing	Stripe — better recurring billing controls for high-value contracts
Conviction subscription at $897/year	Either works. Lemon Squeezy preferred for simplicity.

> *Run both in parallel for your own stack. Lemon Squeezy for the $497 product and the $897 subscription. Stripe for the $10K advisory retainer. Document both integrations in the build repo. The product guide then honestly says: here are both options with their trade-offs, here is the default, here is when to deviate.*

C.2 What to Test and Document

- Lemon Squeezy: successful purchase → GitHub repo invite delivery timing. What is the delay?
- Lemon Squeezy: failed payment. What does the buyer experience? What does your dashboard show?
- Lemon Squeezy: EU buyer. Is VAT correctly applied and shown on receipt?
- Lemon Squeezy: refund flow. How does this affect repo access?
- Stripe: subscription cancellation mid-cycle. What happens to Ghost member status?
- Stripe: webhook to Ghost membership — test the full cycle: signup, payment, Ghost member creation, cancellation, Ghost member status update
- Both: what happens when a buyer uses the system discount (50% off Product 1 as a conviction subscriber)?

D. The Private Intelligence Layer — Build Log

The offline layer is the most personally valuable component of this stack and the hardest to document for others because it is inherently individual. What follows is how to think about building and evolving it for yourself, and what to capture that will eventually inform how you teach it.

D.1 Obsidian Vault Architecture for This Stack

Folder	Purpose

00-inbox/	Unprocessed captures. Everything goes here first. Daily notes. Voice memo transcripts. Screenshots of interesting things. No judgment.
10-conviction/	Processed beliefs. Statements you'd sign your name to. Date-stamped. Tagged. These become posts.
20-decisions/	Pivot documentation. Why you stopped doing X. Why you started doing Y. The record that makes the retrospective honest.
30-tools/	Tool evaluations. Not reviews — personal verdicts. What you actually use, why, and what broke. The raw material for the scoring layer.
40-builders/	Notes on people building interesting things. The Carnegie nuclear researcher. The retired physicist. The displaced professional. The quiet experts.
50-synthesis/	Cross-references. Post drafts. Connections between conviction notes. The layer where the Louvain graph finds its input.
90-archive/	Anything that no longer reflects current thinking but needs to be preserved for the provenance record.

D.2 Ollama Local Model Setup — Stress Test Protocol

The private Ollama layer is only as good as the queries you run against it. Run these weekly as a discipline. Capture outputs in the build repo under ollama-experiments/.

1. What have I written about [topic] in the last 90 days? — Tests corpus recency and tagging quality.
2. What do I actually believe about [contested topic], based only on my own notes? — Tests conviction vs absorbed opinion.
3. What decision have I made that I haven't documented yet? — Often surfaces a 20-decisions/ entry that needs writing.
4. What would [mentor] say about [current decision], based on their published work? — Tests the overlay layer once mentor corpus is indexed.
5. What post am I avoiding writing and why? — Often the most useful query. The Louvain gap + the internal resistance tells you everything about what needs to be said next.

> *The mentor overlay (Robbins, Tolle, Diamandis, Goggins, Novagratz, Master Stephen Co) is indexed from their publicly available books and transcripts. The query is not 'what does Tony Robbins say about X' — it is 'given what Robbins says about peak states and what Goggins says about friction, what does my own history suggest I should do here?' The synthesis is the product, not the citation.*

D.3 Louvain Graph — What to Capture as You Run It

The Louvain script runs weekly. Each run produces a JSON cluster output. Capture the following in louvain-runs/:

- The cluster assignments for all tags — which topics grouped together this week
- The bridge nodes — tags that appear in multiple clusters. These are your synthesis points.
- The isolated nodes — tags with few connections. These are either underwritten topics or topics that don't belong in your corpus.
- The delta from last week — what shifted. A topic moving from one cluster to another is often a signal that your thinking has evolved.
- The gap — which cluster has the fewest recent posts. This is the content calendar in one line.

After six months of runs, the delta log is itself a document — a timestamped record of how your thinking evolved. That record is the kind of thing that cannot be faked retroactively and cannot be produced by someone who hasn't been doing the work. It is evidence.

E. Forward Planning — Privacy, Compliance, DMG, Licensing

These are not current priorities. They are the natural evolution of the current work if the foundation is built correctly. Document them now so the decisions are pre-made when the moment arrives.

E.1 Privacy Architecture Evolution

Stage	What it means
Current (v1)	Self-hosted Ghost on GCP. Data stays on your VM. Backups to Cloud Storage (Google infrastructure). Ollama fully local. This is privacy-respecting but not fully sovereign — GCP has access to your VM.
V2 — encrypted backups	Cloud Storage backups encrypted with a key you hold before they leave the VM. Google cannot read the backup even if they have access to the bucket. One additional line in the backup script.
V3 — sovereign hosting	VM on a non-US provider (Hetzner, OVH, Contabo). GDPR jurisdiction. No Big Tech infrastructure in the stack. Higher ops complexity, appropriate for users with specific regulatory requirements.
V4 — fully local	Ghost running on a Mac M2 via local Docker. No cloud at all. Accessed via Tailscale or Cloudflare Tunnel for remote access. The maximum sovereignty option. This is the DMG target architecture.

> *Document which stage your own stack is at in the build repo. Update it as you evolve. The buyer guide should present V1 as the default and V2–V4 as documented upgrade paths. This is what 'privacy as a spectrum not a binary' looks like in practice.*

E.2 Compliance Considerations — What to Track Now

You are not building a compliance product today. But the architecture decisions you make now either open or close the compliance door later. These are the things to document as you build that will matter if a regulated industry buyer ever comes:

- Every data flow: what enters the system, where it is stored, what leaves. A simple diagram in the build repo. This is the foundation of any data processing agreement.
- GPG key management: where the private key lives, how it is backed up, what happens if it is lost. This is the chain of custody question that compliance teams ask.
- Ghost member data: what PII is stored, where, how long retained. Ghost stores email addresses and subscription status. Document exactly what fields are persisted.
- Payment data: neither Stripe nor Lemon Squeezy stores card data on your server. Document this explicitly — it is the most common compliance concern.
- Backup encryption: document whether backups are encrypted in transit and at rest. Default GCP Cloud Storage: encrypted in transit yes, at rest yes (Google-managed key). Buyer-managed key: add one line to backup script.
- Incident response: what do you do if the VM is compromised? One-page playbook in the build repo. Shut down VM, rotate all credentials, restore from last backup to new VM. That is the whole plan at this scale.

E.3 Patent Strengthening — What This Stack Contributes

The VERA provisional patent covers the verification architecture. Every documented implementation of that architecture — in your own stack, in buyer installs, in the build repo — strengthens the prior art record and the non-provisional filing.

What to document	Why it matters for the patent
The GPG-signed commit on Ghost publish	Live implementation of VERA's session certificate concept applied to publishing. Timestamped, signed, attributable.
The Louvain conviction scoring	The offline ranking algorithm as a personal application of the evidence tier classification system. Dated runs = dated prior art.
The three-layer content architecture	Public / gated / private as an implementation of VERA's sovereignty audit framework at the individual level.
The payment + access control integration	License key → private repo access as a minimal implementation of the gated knowledge layer.

The Ollama local query layer	Private inference over personal corpus as an implementation of the offline-first intelligence principle.

> *Every dated commit in the build repo is prior art. Every published post that describes the architecture is prior art. Every buyer install that documents a working implementation is prior art. The non-provisional patent filing in March 2027 will be stronger for every month of documented implementation between now and then.*

E.4 DMG — The Right Conditions to Build It

The DMG (a Mac-native installer that walks a buyer through the full sovereign stack) is the right long-term product form factor. It is not the right current priority. These are the conditions that make it worth building:

Condition	What it means
20+ manual installs completed	Enough real installs to know exactly which steps break and why. The DMG hardens the manual process — you cannot harden what you have not run.
Failure mode library complete	The build repo's failure-modes/ folder has at least 15 documented failure modes with exact fixes. This becomes the DMG's error handling.
The monthly maintenance cycle is boring	If running 'ghost update' on your own stack is completely uneventful for three consecutive months, the automation is stable enough to ship.
A buyer has asked for it specifically	Market demand is confirmed, not assumed. One buyer who would have paid $50 more for a DMG install is enough signal to start scoping.
The V4 local architecture is tested	Mac M2 running Ghost locally via Docker, accessed via Cloudflare Tunnel. Test this on your own machine first. If it runs stably for 30 days, the DMG architecture is proven.

Technical path when the conditions are met

- The DMG is a shell script wrapped in an Automator app or a minimal Electron/Tauri shell. No complex GUI required.
- The script installs Homebrew if missing, then: gcloud CLI, Ghost CLI, Ollama, and the provenance sidecar from the private repo.
- Interactive prompts: GCP project ID, domain name, Mailgun or Brevo API key, Lemon Squeezy or Stripe key. Nothing else.

- The script ends by opening the buyer's Ghost admin in a browser. One confirmation message: your sovereign stack is live.
- The failure mode library becomes the script's error handling: each known failure has a specific error code and a plain-English resolution message.
- Code signing via Apple Developer Program ($99/year) so the DMG passes Gatekeeper. Required for distribution outside the App Store.

E.5 Licensing — How to Think About It

The product system repo (delivered to buyers) is proprietary — the buyer purchases a license to use it, not to redistribute it. The VERA public documentation repo is open source (GPL or CC-BY). The Ghost provenance sidecar script is a candidate for open source release once it is stable — open sourcing it creates a citable GitHub artifact that strengthens both the VERA patent record and the product's authority.

Component	License approach
Product 1 system repo (private)	Proprietary. Lemon Squeezy license key grants personal use. No redistribution. No sublicensing.
VERA public documentation	Creative Commons CC-BY 4.0 or GPL v3. Attribution required. Freely usable and citable.
Ghost provenance sidecar	Open source (MIT) once stable. The 'smallest useful piece' that demonstrates VERA in practice. Citable, forkable, creates upstream pull toward the full product.
Louvain conviction script	Open source (MIT) once validated. Same logic — the methodology published openly strengthens the authority of the system that applies it.
Ollama mentor corpus queries	Not a licensable artifact — these are personal query patterns. Document them in the build repo for your own reference.
DMG installer (future)	Proprietary. Part of the v2 product at a higher price point. Not open source — the value is in the hardening and the error handling, not the shell script itself.

F. The Conviction Layer — Why This Is Different From Every Other Platform

A platform gives you an audience. A domain gives you a memory. The distinction is not rhetorical — it is architectural.

Platforms are optimized for engagement within their system. The metric that matters to them is time-on-platform. Your posts, your subscribers, your history — all of it exists to serve that metric.

When you leave, the history stays. When the platform changes its algorithm, your visibility changes. When the platform dies, everything you built there dies with it.

A domain under your name is optimized for one thing: the accumulation of a verifiable record of who you are and what you know. The metric that matters is the quality of the timestamp. A post published on your domain on March 15th, 2025, with a GPG-signed commit, a JSON-LD author attribution, and a Louvain graph node — that post exists in a form that is yours regardless of what any platform does. The record predates any claim anyone else might make about having had the same idea.

A person will have something that is dynamic, evolves over time, is validated, and has a memory outside of algorithms.
That is the product. Everything else is infrastructure.

The people this is built for are not looking for followers. They are looking for proof — first for themselves, then for anyone who needs to know who they are. The incognito browser user. The person who turns off ad tracking. The professional who has been meaning to ‘get their name out there’ for three years but hasn’t because every option has felt like a performance they weren’t willing to give. The retired expert who has something worth saying and no desire to compete with influencers to say it.

This stack exists for that person. It is not for everyone. The conviction component is the most important component and it gets built over time — one signed post, one dated commit, one honest reflection at a time. The person who does this for a year has something that cannot be purchased, replicated, or extracted: a verified record of a specific mind working through specific problems in public, under their own name, before anyone told them to.

That record is the universal high income infrastructure. Not because it guarantees income — because it makes income possible from a foundation that is entirely theirs.